УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ
КАТЕДРА ЗА РАЧУНАРСТВО

Семинарски рад

Анализа HTTP саобраћаја из PCAP и PCAPNG фајлова у сврху откривања поверљивих података применом Python алата

Ментор:
проф. др Братислав Предић

Студент:
Александар Цајић
Бр.индекса: 2082

Ниш, фебруар 2026. год.

Сажетак

У оквиру овог рада развијен је форензички софтверски алат намењен анализи унапред снимљеног мрежног саобраћаја у pcap и pcapng форматима, са циљем аутоматске идентификације и екстракције осетљивих података који се преносе нешифрованим HTTP протоколом. Мотивација за овај рад је из разлога што се и даље срећу веб апликације које користе неадекватно заштићене комуникационе канале, чиме се отвара могућност за пресретање података и касније злоупотребе.

Алат је урађен помоћу програмског језика Python и коришћена је PyShark библиотека за парсирање мрежних пакета. Алат омогућава учитавање pcap и pcapng записа из директоријума и њихову аутоматску обраду, чиме се постиже брза обрада снимљеног саобраћаја.

Оно што се постиже са овим алатом јесте обрада HTTP POST захтева тако што се декодира тело поруке и парсирају се подаци послати путем веб форми, а ту можемо да сретнемо имена, лозинке и друге параметре. Поред POST и HTTP GET захтеви се анализирају и издвајају се параметри прослеђени кроз URL упите. Такође је имплементирано издвајање HTTP Basic аутентификације декодирањем Base64 креденцијала као и идентификација савремених аутентификационих токена као што су Bearer токени из authorization заглавља и токене присутне у телу захтева. Поред тога ради се и анализа HTTP колачића тако да се разликују колачићи које поставља сервер и оне које клијент враћа.

На крају након анализе снимљеног саобраћаја резултати анализе се приказују кроз генерисани форензички извештај у одговарајућем формату који је погодан да корисник види који типови података се налазе у анализираним записима.

Садржај

УВОД **4**

Шта је дигитална форензика **4**
Значај мрежне форензике **4**
Мотивација за израду рада **5**
Циљеви рада **5**

HTTP ПРОТОКОЛ **6**

HTTP request и response **6**
GET и POST методе **7**
HTTP заглавље **7**
Content-Type **7**
Cookies **8**
Authorization header **8**
Аутентификација у HTTP-у **8**
Basic Authentication (Basic Auth) **9**
Bearer token **9**
JSON Web Token (JWT) **9**
Session cookies **10**

НАЧИН ИЗРАДЕ ПРОЈЕКТА **10**

Прикупљање мрежног саобраћаја **10**
Дефинисање критеријума детекције осетљивих података **10**

ИМПЛЕМЕНТАЦИЈА **11**

Учитавање PCAP/PCAPNG фајлова **11**
Екстракција HTTP POST захтева **11**
Екстракција HTTP GET параметара **12**
Ekstrakcija Basic Authentication **12**
Ekstrakcija токена **13**
Ekstrakcija колачића **14**
Парсирање форм дата **14**

ОГРАНИЧЕЊА И УНАПРЕЂЕЊЕ АПЛИКАЦИЈЕ **16**

ЗАКЉУЧАК **17**

ЛИТЕРАТУРА **18**

Увод

Савремене информационе системе сусрећемо свуда током свакодневног пословања и комуникације, што их чини честом метом различитих безбедносних инцидената. Напади на рачунарске мреже, неовлашћен приступ системима и цурење поверљивих података представљају озбиљан проблем за организације и појединце. У таквом окружењу јавља се потреба за апликацијама које омогућавају идентификацију, анализу и разумевање дигиталних инцидената.

У разумевању ових инцидената главну улогу у овом процесу играју дигитална и мрежна форензика, јер омогућавају анализу дигиталних трагова насталих током комуникације и рада информационих система. Посебан значај има анализа мрежног саобраћаја, која може пружити увид у догађаје чак и након што су компромитовани системи враћени у првобитно стање.

Шта је дигитална форензика

Дигитална форензика је област форензичких наука која се бави прикупљањем, очувањем, анализом и интерпретацијом дигиталних доказа. Дигитални докази могу потицати са различитих извора, као што су рачунари, сервери, мобилни уређаји, мрежни системи и апликације.

Циљ дигиталне форензике је да се вратимо у прошлост у информационом систему и урадимо идентификацију неовлашћених активности и утврђивање узрока безбедносних инцидената. Посебна пажња се посвећује очувању валидности доказа, како би резултати анализе били поуздани и употребљиви у техничком и правном контексту. [1]

Значај мрежне форензике

Мрежна форензика представља поддисциплину дигиталне форензике која се фокусира на анализу мрежног саобраћаја и комуникацију између рачунарских система. Мрежна

форензика омогућава увид у токове података који су се одвијали између уређаја, независно од стања самих система.

Значај мрежне форензике посебно долази до изражаја у анализи безбедоносних инцидената, јер мрежни саобраћај често садржи информације о нападима, неовлашћеним приступима и преносу осетљивих података. Оно што је могуће постићи анализом мрежних пакета је идентификовање сумњиве активности, компромитоване комуникације и потенцијално цурење информација. [2]

Мотивација за израду рада

У данашњем времену где има много информационих система са којима се сусрећемо у свакодневном животу присутан је пораст броја безбедносних инцидената у рачунарским мрежама, а и честа је појава цурење осетљивих података, што нам указује на потребу за ефикаснијим методама анализе мрежног саобраћаја. Постоје бројни алати за анализу мрежних пакета, али за њихову употребу треба искуства, а ручно анализирање може бити временски захтевно и подложно грешкама.

Посебан проблем представља HTTP саобраћај који се због своје природе и присутности у старијим и лоше конфигурационим системима и даље користи за пренос осетљивих информација у нешифрованом облику. Такав саобраћај представља значајан безбедносни ризик, јер омогућава лако пресретање и злоупотребу података од стане неовлашћених лица. Па ако узмемо ово у обзир мотивација за израду овог рада проистиче из потребе за аутоматизованим приступом анализи pcap и pcapng фајлова, са циљем брзе идентификације потенцијално компромитујућих информација у мрежном саобраћају. Развој специјализованог алата за анализу HTTP комуникације може значајно допринети ефикаснијој форензичкој анализи и бољем разумевању безбедоносних инцидената.

Циљеви рада

Основни циљ овог рада јесте развој и примена алата за форензичку анализу мрежног саобраћаја, са посебним фокусом на HTTP протокол и pcap и pcapng фајлове. У оквиру овог рада дефинисани су следећи специфични циљеви:

- Анализа pcap и pcapng фајлова који садрже мрежни саобраћај
- Идентификација и екстракција HTTP захтева и одговора
- Аутоматска детекција осетљивих података, као што су корисничко име, лозинке, аутентификациони токени и сесијски колачићи
- Идентификација потенцијалних безбедносних пропуста насталих услед коришћења нешифрованог HTTP саобраћаја

HTTP протокол

Hypertext Transfer Protocol (HTTP) је апликациони протокол који се користи за размену података између клијента и сервера у оквиру рачунарских мрежа. Најчешће се користи за комуникацију веб прегледача са веб серверима, али и за размену података између различитих апликација и сервиса. HTTP је по својој природи stateless протокол, што значи да сваки захтев представља независну целину и не садржи информације о претходним захтевима. Због тога се као додатни механизам за одржавање стања између клијента и сервера користе колачићи и сесије. [3]

HTTP request и response

HTTP комуникација се заснива по моделу захтева и одговора, клијент шаље захтев ка серверу док сервер тај захтев обрађује и враћа одговор.

HTTP request се састоји од:

- Почетне линије која садржи методу, путању ресурса и верзију протокола
- HTTP заглавље(headers)
- Опционог дела поруке (body)

HTTP response садржи:

- Статусну линију са статусним кодом одговора
- Заглавље
- Тело поруке које садржи тражени садржај или поруку о грешци

Због овакве структуре можемо да извршимо детаљну анализу комуникације приликом форензичке обраде мрежног саобраћаја. [4]

GET и POST методе

GET и POST методе су најчешће коришћене HTTP методе за комуникацију између клијента и сервера.

GET метода се користи за захтев ресурса са сервера, при чему се параметри захтева често прослеђује кроз URL. Због тога су подаци послати GET методом видљиви у адресној линији прегледача и лако доступни током анализе мрежног саобраћаја.

POST метода се користи за слање података ка серверу најчешће кроз тело HTTP захтева. Ова метода се често користи за слање података са форми, укључујући корисничка имена и лозинке. [4]

HTTP заглавље

HTTP заглавља представљају кључ – вредност парове који садрже додатне информације о захтеву или одговору. Она омогућавају пренос метаподатака, као што су информације о клијенту, формату података, аутентификацији и стању сесије. Заглавља имају посебан значај у форензичкој анализи јер често садрже осетљиве информације које могу указивати на компромитацију система или неадекватну конфигурацију апликације. [5]

Content-Type

Заглавље Content-Type дефинише тип података који се шаљу у телу HTTP захтева или одговора. Оно омогућава серверу и клијенту да правилно интерпретирају садржај поруке. Примери често коришћених вредности укључују text/html, application/json и application/x-www-form-urlencoded. Ако гледамо у контексту форензике, Content-Type заглавље је важно јер омогућава идентификацију формата података, што олакшава екстракцију корисничких уноса, креденцијала и других релевантних информација. [6]

Cookies

HTTP колачићи представљају мале количине података које сервер шаље клијенту, а које клијент затим враћа у сваком наредном захтеву ка истом серверу. На овај начин омогућава се одржавање сесије и идентификације корисника. Колачићи често садрже сесијске идентификаторе или токене, који уколико се преносе нешифровано, могу бити пресретнути и злоупотребљени. Због тога колачићи представљају један од најважнијих елемената за анализу у мрежној форензици. [7]

Authorization header

Заглавље Authorization користи се за аутентификацију клијента према серверу. Оно може садржати различите типове података као што су основна аутентификација (Basic Auth), токени или API кључеве. У случају нешифроване HTTP комуникације садржај овог заглавља може лако бити доступан нападачу што представља озбиљан безбедоносни ризик. Анализа Authorization заглавља у pcap и pcapng фајловима омогућава идентификацију небезбедних механизама аутентификације и потенцијално компромитованих креденцијала. [8]

Аутентификација у HTTP-у

Аутентификација у HTTP протоколу представља процедуру којом се потврђује дати клијент приликом приступа заштићеним ресурсима на серверу. Пошто је HTTP stateless протокол, аутентификација се најчешће реализује помоћу заглавља или додатних података који се прослеђују у сваком захтеву. Различити механизми аутентификације имају различите безбедоносне карактеристике, а њихова неправилна имплементација може довести до озбиљних безбедоносних пропуста. [9]

Basic Authentication (Basic Auth)

Basic Authentication је један од најједноставнијих механизама аутентификације у HTTP протоколу. Код овог приступа корисничко име и лозинка се комбинују у један стринг, који се затим кодира помоћу Base64 алгоритма и шаље серверу кроз Authorization заглавље. Basic Auth је врло једноставан за имплементацију али такође не пружа стварну енкрипцију података јер дати алгоритам врши само кодирање а не и заштиту података. У случају коришћења нешифрованог HTTP саобраћаја, креденцијали могу бити лако пресретнути што овај механизам чини небебедним без употребе HTTPS-а. [10]

Bearer token

Аутентикација заснива се на приципу да клијент поседује токен који доказује његов индентитет. Токен се шаље у Authorization заглављу, најчешће у облику Authorization: Bearer token. Сервер прихвата захтев уколико је токен валидан без додатне провере идентитета корисника. Због тога се Bearer токени сматрају осетљивим подацима односно свако ко поседује валидан токен може се представити као ауторизовани корисник. У нешифрованом HTTP саобраћају пресретање токена омогућава нападачу потпун приступ заштићеним ресурсима. [11]

JSON Web Token (JWT)

JSON Web Token (JWT) представља специфичну врсту Bearer токена који садржи структурисане податке у json формату. JWT се састоји од три дела: заглавља, тела и дигиталног потписа. JWT омогућава серверу да провери аутентичност токена без потребе за чувањем стања сесије. Оно што је проблем у JWT-у што су подаци углавном само Base64 кодирани а нису енкриптовани тако да се пресретањем података може бити откривен њихов садржај. У форензичком контексту JWT токени у мрежном саобраћају могу открити идентитет корисника, важење токена и друге осетљиве податке. [12]

Session cookies

Сесијски колачићи представљају један од најчешће коришћених механизама аутентификације у веб апликацијама. Након успешне пријаве сервер клијенту додељује јединствени идентификатор сесије који се чува у колачићу и шаље у сваком наредном HTTP захтеву. Уколико се сесијски колачићи преносе нешифровано нападач може извести крађу сесије и преузети идентитет корисника. Због тога сесијски колачићи представљају кључну тачку интересовања у мрежној форензици. [13]

Начин израде пројекта

Прикупљање мрежног саобраћаја

Прикупљање мрежног саобраћаја представља основу за истраживање и даљу форензичку анализу. Снимање је урађено у локалном тестном окружењу коришћењем алата Wireshark, који омогућава детаљно пресретање саобраћаја и бележење мрежне комуникације на нивоу пакета.

У циљу обезбеђивања узорака на којима можемо да применимо имплементирани алат направљени су наменски HTTP захтеви који симулирају реалне корисничке активности. Током снимања саобраћаја обухваћене су следеће врсте комуникације:

- HTTP GET захтеви са параметрима у URL адреси
- HTTP POST захтеви са подацима у телу поруке
- Слање аутентификационих заглавља
- Размена колачића између клијента и сервера

Након завршетка снимања саобраћај је сачуван у одговарајућем формату који омогућава чување детаљних метаподатака о времену, интерфејсу и структури пакета.

Дефинисање критеријума детекције осетљивих података

Један од кључних корака у изради алата је дефинисање критеријума на основу којих се одређени подаци дефинишу као осетљиви. Ови критеријуми се заснивају на

безбедоносним принципима и уобичајеним обрасцима преноса података у веб апликацијама. У овом алату под осетљивим подацима се подразумевају:

- Аутентификациони подаци
- Параметри за пријаву
- Колачићи и сесиони идентификатори
- Токени

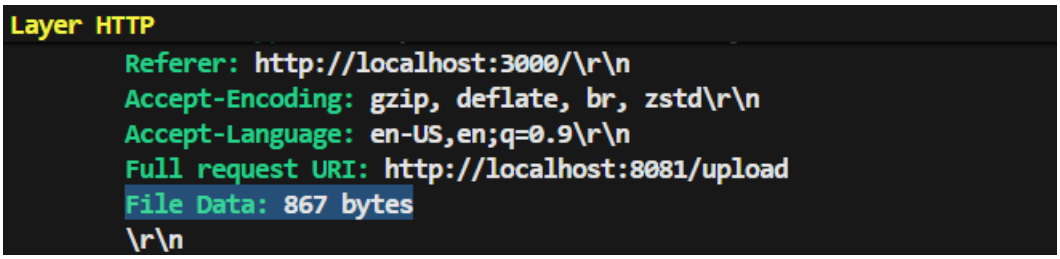
Имплементација

Учитавање PCAP/PCAPNG фајлова

Функција за учитавање пролази кроз директоријум и бира само фајлове са екстензијом .pcap и .pcapng. PyShark користи Tshark и предност је што се већ добије декодирани протокол и поља (нпр. packet.http.host, packet.http.authorization) без ручног парсирања. Улога display_filter служи да Tshark већ на нивоу кад извлачи податке избаци небитне пакете. Ако филтер није постављен добијају се сви пакети што отежава анализу HTTP. Функција враћа листу парова (filename, capture) што омогућава main-у да зна из ког фајла долазе пакети.

Екстракција HTTP POST захтева

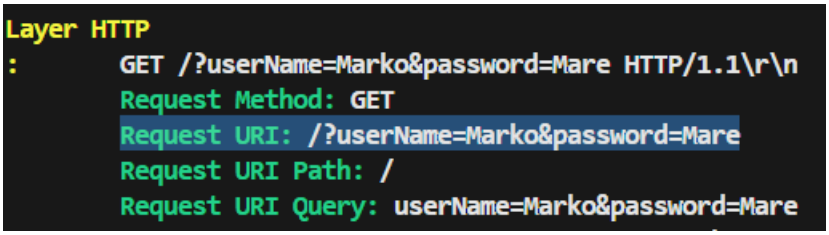
Ова функција ради следећу валидацију: проверава да ли пакет има HTTP слој, проверава да ли HTTP има поље request_method, проверава да ли је метода POST, а затим проверава да ли постоји file_data односно payload који Tshark издваја. Tshark често враћа тело POST захтева као хексадецимални низ, форматиран са двотачкама (нпр. 61:62:63..). Зато функција уклања :, претвара hex у “raw bytes” и декодира bytes у стринг. После тога извлачи и content_type, јер од њега зависи како ће тело поруке бити парсирано (json, urlencoded, multipart). POST захтеви су битни јер често носе : корисничко име, лозинку у форми, токен у тело ...



Слика 1 – Приказ HTTP слоја и податка file_data

Екстракција HTTP GET параметара

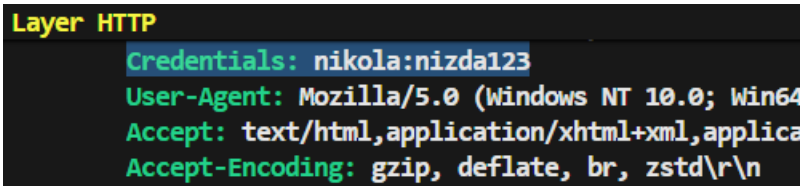
GET захтеви се анализирају другачије осетљиви подаци нису у телу захтева већ у URL-у. Функција проверава следеце ствари : проверава да постоји HTTP слој и requeset_method, онда проверава да ли је метода GET, затим узима request_uri и парсира га помоћу urlparse, а затим се из parsed.query извлаче параметри преко parse_qs. parse_qs враћа мапу где свака вредност може бити листа (јер параметар може да се појави више пута). Функција враћа резултате само ако постоји бар један параметар – тиме избегава празне GET захтеве. GET параметри су чест извор цурења података јер остају у логовима сервера, виде се у историји browsera ...



Слика 2 – Приказ HTTP слоја и request_uri одакле се извлаче параметри

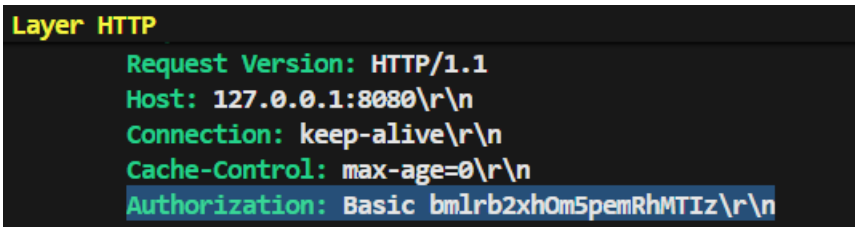
Ekstrakcija Basic Authentication

Овај модул решава реалан проблем : Basic Auth може бити изложен на два начина у PyShark/Tshark izlazu и то ипситивањем http.credentials и http.authorization. Што се тиче http.credentials ако Tshark препозна Basic Auth он изгледа у облику username:password, а то је случај где је довољно урадити само split по : .



Слика 3 – Пиказ HTTP слоја и како је податак записан у credentials

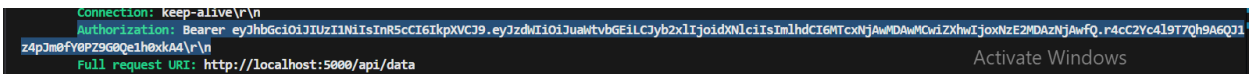
Други случај је ако нема credentials проверава се authorization хедер. Ако почиње са “Basic” остатак је Base64 стринг. Функција га декодира у текст и опет се добија формат username:password. Оно што је битно јесте да Basic Auth није шифрован већ је само Base64 кодиран, а то значи да се може лако урадити реконструкција креденцијала без икакве додатне опреме.



Слика 4 – Приказ HTTP слоја и како је податак уписан у Authorization header

Ekstrakcija tокена

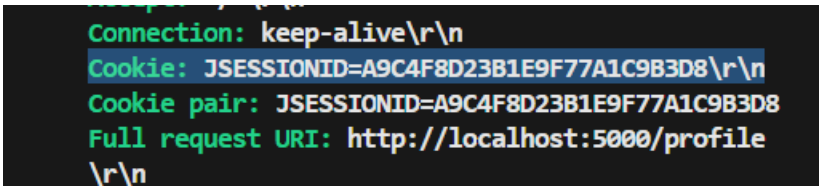
Овај алат покрива две главне класе токена и то су Bearer токен у Authorization хедеру и токен у телу захтева. Ако постоји http.authorization и почиње са Bearer функција само узима остатак стринга као токен. Други део претражује file_data у POST телу и ту можемо видети кључне речи као што су : access_token, token, jwt. Оно због чега су битни токени јесте то да се приступ систему може обавити преко њих без лозинке. Ако нападач пресретне токен из нешифрованог саобраћаја може се преузети сесија или позвати API у име корисника.



Слика 5 – Приказ HTTP слоја и Bearer токена

Ekstrakcija kolačića

Овде имамо две врсте колачића: прва врста су они који клијент шаље серверу, а други обрнуто. Прва врста садржи Cookie хедер и parse ради тако што дели стринг по ; , а затим сваку ставку дели по = и на крају памти вредности name и value. Друга врста је она која садржи set-cookie хедер и то сервер шаље клијенту. Сервер може да шаље један или више set-cookie хедер па се зато проверава да ли је set-cookie листа или стринг. Из сваког тог колачића узима се први део пре ; , јер су после атрибути (HttpOnly, Path, Expires). Резултати се обележавају пољем direction: request(cookie) и response(set-cookie).



Слика 6 – Приказ HTTP слоја и колачића

Парсирање форм дата

Поента овог модула је да се претвори “raw body” у структуриране податке које се касније могу анализирати и направити извештај о тим подацима.

JSON

Ako je Content-Type: application/json, pokušava se json.loads. Ako ne uspe, vraća se prazno (što sprečava pad programa).

x-www-form-urlencoded

Ovo je klasičan format HTML formi (a=b&c=d). urllib.parse.parse_qs ga pretvara u mapu.

multipart/form-data

Ovo je složen format (file upload, forme sa boundary). Kod radi minimalno parsiranje:

- izvlači boundary iz Content-Type
- deli telo poruke po boundary markeru
- za svaki deo razdvaja headere i sadržaj (\r\n\r\n)
- iz headera izvlači name="..."

Ограничења и унапређење апликације

Најзначајније ограничење алата огледа се у чињеници да је анализа фокусирана искључиво на нешифровани HTTP протокол. Сада код савремених апликација најчешће се користи HTTPS односно TLS енкриптована комуникација чији садржај није могуће анализирати без одговарајућих криптографских кључева.

Зависност од библиотеке PyShark која се користи за издвајање података декодирањем протокола. Такође у одређеним случајевима нека HTTP поља можда неће бити доступна или ће бити парсирана другачије.

Анализа великих снимака мрежног саобраћаја може довести до потрошње системских ресурса.

Најзначајније унапређење би се односило на имплементацију механизма за анализу HTTPS саобраћаја и то би укључивало: TLS кључ декрипцију, анализу handshake метаподатака.

Такође апликацију је могуће унапредити додавањем метода детекције попут: регуларних израза за кредитне картице и телефоне, детекцију личних индентификационих бројева, примену машинског учења за класификацију података.

Тренутна имплементација заснива се на снимљени мрежни саобраћај. Додавањем модула који би анализирао саобраћај у реалном времену омогућила би се бржа детекција безбедоносних инцидената.

Имајући у виду ограничења и могуће унапређење може се рећи да развијени алат представља функционалну основу за аутоматизовано анализирање нешифрованог саобраћаја, али да такође је и погодан за даље истраживање и унапређивање.

Закључак

У оквиру овог рада виђено је да анализа унапред снимљеног мрежног саобраћаја представља ефикасан приступ у идентификацији осетљивих података, а развијени софтверски алат демонстрира могућност аутоматизованог издвајања осетљивих података из нешифрованог HTTP саобраћаја чиме се смањује потреба за ручном анализом великог броја пакета.

Из овог рада се може видети да велики број важних података може се идентификовати већ на нивоу апликационог слоја комуникације. Уочено је да се аутентификациони подаци, параметри веб форми, токени и колачићи могу прочитати и забележити директно из мрежних записа уколико се комуникација одвија путем нешифрованог протокола. Ово нам показује да постоји висок ризик од крађе података у окружењу где се HTTP користи без додатне заштите.

Треба нагласити да домет алата остаје на нешифровани саобраћај. Савремени информациони системи у значајном броју користе HTTPS и друге криптографски заштићене протоколе што отежава директну анализу садржаја комуникације. Тако да даљи развој алата треба усмерити ка подршци за TLS декрипцију, анализу метаподатака шифорваног саобраћаја и доградњу напреднијих механизама за детекцију осетљивих података.

На основу резултата које смо добили у раду долазимо до закључка да аутоматизација анализе мрежних записа знатно олакшава обраду мрежног саобраћаја, али и да истовремено указује на неопходност примене енкрипције у савременим комуникационим системима, како би се умањили ризици од неовлашћеног пресретања и злоупотребе осетљивих података.

Литература

[1] - What is digital forensics? | IBM

[2] - What is Network Forensics? - GeeksforGeeks

[3] - Overview of HTTP - HTTP | MDN

[4] - HTTP Requests & Responses Explained : A Beginner's Guide

[5] - HTTP headers - HTTP | MDN

[6] - Content-Type header - HTTP | MDN

[7] - Using HTTP cookies - HTTP | MDN

[8] - Understanding the Authorization Header in HTTP | BrowserStack

[9] - Explain HTTP authentication - GeeksforGeeks

[10] - What is Basic Authentication? ✓

[11] - Bearer Token Authentication: The Complete Guide for Developers - DEV Community

[12] - JSON Web Token (JWT) - GeeksforGeeks

[13] - Session Cookies: From Basics to Best Practices | Transcend | The compliance layer for customer data